

Fiber Data Structures

Overview

Fiber is a data structure introduced in React 16 as part of a complete rewrite of React's reconciliation algorithm. It represents a unit of work and enables React to break rendering work into chunks, pause and resume work, assign priority to different types of updates, and reuse previously completed work.

What is a Fiber?

A fiber is a JavaScript object that contains information about a component, its input, and its output. Each fiber corresponds to a stack frame, but also corresponds to an instance of a component.

Core Concept

Think of a fiber as a virtual stack frame that React can pause, resume, or discard. Unlike the actual JavaScript call stack, fibers give React control over the execution flow, allowing it to:

- Pause work and come back to it later
- Assign priority to different types of work
- Reuse previously completed work
- Abort work if it's no longer needed

Fiber Node Structure

Each fiber node contains several key properties:

```

{
  // Identity
  type: Function | String | Symbol, // Component type (function, class, or host element)
  key: string | null,                // Unique identifier

  // Relationships (forming a tree)
  return: Fiber | null,              // Parent fiber
  child: Fiber | null,               // First child fiber
  sibling: Fiber | null,              // Next sibling fiber

  // State
  pendingProps: any,                // New props from parent
  memoizedProps: any,               // Props used for previous render
  memoizedState: any,               // State used for previous render
  updateQueue: UpdateQueue | null,  // Queue of state updates and callbacks

  // Effects
  flags: Flags,                     // Side-effect tags (placement, update, deletion)
  subtreeFlags: Flags,              // Flags for the entire subtree
  deletions: Array<Fiber> | null,   // Child fibers to delete

  // Work tracking
  lanes: Lanes,                     // Priority of this work
  childLanes: Lanes,                // Priority of child work

  // Double buffering
  alternate: Fiber | null,           // Work-in-progress / current fiber pair

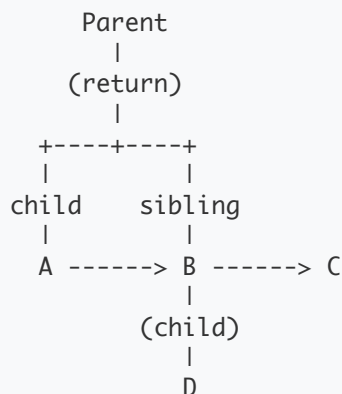
  // Output
  stateNode: any,                   // Reference to component instance, DOM node, etc.
}

```

The Fiber Tree Structure

Tree Relationships

Fibers form a tree using three pointers:



- **child:** Points to the first child
- **sibling:** Points to the next sibling
- **return:** Points to the parent (called "return" because it's where we return after processing this fiber)

Tree Traversal

React traverses the fiber tree using depth-first search:

```
function performUnitOfWork(fiber) {
  // 1. Work on current fiber
  // ...

  // 2. If there's a child, go there next
  if (fiber.child) {
    return fiber.child;
  }

  // 3. If no child, check for sibling
  let nextFiber = fiber;
  while (nextFiber) {
    if (nextFiber.sibling) {
      return nextFiber.sibling;
    }
    // 4. If no sibling, go up to parent
    nextFiber = nextFiber.return;
  }

  return null; // Reached root
}
```

Double Buffering (Current and Work-in-Progress)

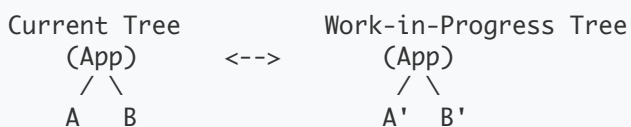
React maintains two fiber trees:

Current Tree

- Represents what's currently on screen
- Stable and reflects the last committed state

Work-in-Progress Tree

- The tree being built during rendering
- Contains updates and changes being processed



These trees are connected via the `alternate` pointer, allowing React to:

- Preserve the current state while building the next state
- Quickly swap trees during commit phase
- Reuse fiber objects between renders (memory optimization)

How Fibers Enable Concurrent Rendering

Work Loop

React's work loop can be interrupted:

```
function workLoop(deadline) {
  while (nextUnitOfWork && deadline.timeRemaining() > 0) {
    nextUnitOfWork = performUnitOfWork(nextUnitOfWork);
  }

  if (nextUnitOfWork) {
    // More work to do, schedule continuation
    requestIdleCallback(workLoop);
  } else {
    // Work complete, commit changes
    commitRoot();
  }
}
```

Priority Lanes

Fibers use a lanes system for priority:

```
// Example lane priorities (simplified)
const SyncLane = 0b0001; // Highest: user interactions
const InputContinuousLane = 0b0010; // Text input
const DefaultLane = 0b0100; // Normal updates
const TransitionLane = 0b1000; // Low priority: transitions
```

Higher priority work interrupts lower priority work, ensuring responsive UIs.

Lifecycle of a Fiber

1. Creation

```
function createFiber(type, props, key) {
  return {
    type,
    key,
    props,
    // ... other properties initialized
  };
}
```

2. Reconciliation

- Compare old and new props
- Determine what changed

- Mark with appropriate flags (update, placement, deletion)

3. Commit

- Apply all side effects to the DOM
- Call lifecycle methods
- Update refs

4. Cleanup

- Fibers are reused via the alternate pointer
- Memory efficient as objects are recycled

Effect Flags

Fibers track side effects with bit flags:

```
const Placement = 0b00001; // Insert into DOM
const Update = 0b00010; // Update existing node
const Deletion = 0b00100; // Remove from DOM
const Ref = 0b01000; // Update ref
const Callback = 0b10000; // Has callback to run
```

React can efficiently check multiple flags using bitwise operations:

```
if (fiber.flags & (Placement | Update)) {
  // Has placement or update
}
```

Update Queue

Each fiber maintains an update queue for state changes:

```
{
  updateQueue: {
    baseState: any, // State before first update
    firstBaseUpdate: Update | null,
    lastBaseUpdate: Update | null,
    shared: {
      pending: Update | null, // Circular linked list of updates
    },
    effects: Array<Update> | null,
  }
}
```

Updates form a circular linked list, processed in order during rendering.

Practical Example

When you call `setState`:

```
function Component() {
  const [count, setCount] = useState(0);

  const handleClick = () => {
    setCount(count + 1); // This creates an update
  };

  return <button onClick={handleClick}>{count}</button>;
}
```

What happens:

1. **Update created:** An update object is created and added to the fiber's update queue
2. **Work scheduled:** React schedules work with appropriate priority
3. **Render phase:**
 - Work-in-progress fiber is created (or reused from alternate)
 - Update queue is processed
 - New state is calculated
 - Fiber is marked with Update flag
4. **Commit phase:**
 - DOM is updated
 - `count` changes from 0 to 1
 - Current and work-in-progress trees are swapped

Benefits of Fiber Architecture

1. Interruptible Rendering

Break work into small units, pause and resume as needed

2. Priority-Based Updates

Handle urgent updates (clicks) before less urgent ones (data fetching)

3. Better Error Handling

Error boundaries can catch errors and handle them gracefully

4. Concurrent Features

- Suspense for data fetching
- Transitions for non-urgent updates

- Time slicing for smooth user experiences

5. Memory Efficiency

Reuse fiber objects between renders via double buffering

Key Takeaways

- **Fiber is a unit of work:** Each fiber represents work to be done for a component
- **Tree structure:** Fibers form a tree using child, sibling, and return pointers
- **Double buffering:** Two trees (current and work-in-progress) enable smooth updates
- **Priority system:** Lanes allow React to prioritize important updates
- **Interruptible:** Work can be paused, resumed, or aborted
- **Effect tracking:** Flags efficiently track what changes need to be made

The fiber architecture is the foundation that enables React's concurrent features and provides a smooth, responsive user experience even during complex updates.